



CSE Department – Faculty of Engineering - MSA

Spring 2025

GSE122 GSE122i COM265 PROGRAMMING 2

Course Project

Course Instructor. Ahmed El Anany

Due Date 17/MAY/2025 11:59 PM on E-learning**Discussion inside lecture 18/May till 23/May inside lab as per lab slot**

Student Name	Gamila Hassan	Student ID	242127
Student Name	Laila Zulfacar	Student ID	243985
Student Name	Mariam Mohamed Reyad	Student ID	245171
Student Name	Marina Ayman	Student ID	246635
Student Name	Tasbeeh Khater	Student ID	240909
TA Name	Eng. Dina Magdy Eng. Youmna Mohamed Eng. Mohamed Khaled Eng. Hussien Mostafa	Grade:	/

Library Management System



Table of Contents

Project Overview	3
Objectives	3
Roles and Responsibilities	4
Algorithm and external libraries	5
GUI and Database Usage	6
Code explaining	7
Output and results	8
References	9



Project Overview

Objectives

The library management system acts as a catalogue where the user can view, search, issue, edit, or delete a book. It provides a user-friendly interface which has quick access and an organized structure. The user is only required to press a few buttons and type out the title of the desired book.

Roles and Responsibilities

- Database Handler: Tasbeeh Khater
- GUI: Gamila Hassan and Laila Zulfacar
- Book Manager: Marina Ayman
- Student Record Manager: Mariam Reyad



Algorithm and external libraries

1. `import java.io.File;`

- It can check if a file exists, create a new file, or get the name and path of a file

2. `import java.util.ArrayList;`

- Stores a list of books in the memory
- Allows the user to add new books and search through existing ones
- User can remove or update books

3. `import java.io.FileWriter;`

- Saves book information into text files

4. `import java.io.IOException;`

- Catches errors when the program is running

5. `import javax.swing.*;`

- Gives the user access to Swing GUI components (JFrame, JPanel, JButton, JLabel, JOptionPane, etc)

6. `import java.awt.*;`

- Provides the user with access to layout managers and styling elements (background and text colors, font types and sizes, etc)

7. `import java.awt.event.ActionEvent;`

- Stores information about events performed by the user (closing a window, pressing a button, etc)

8. `import java.awt.event.ActionListener;`

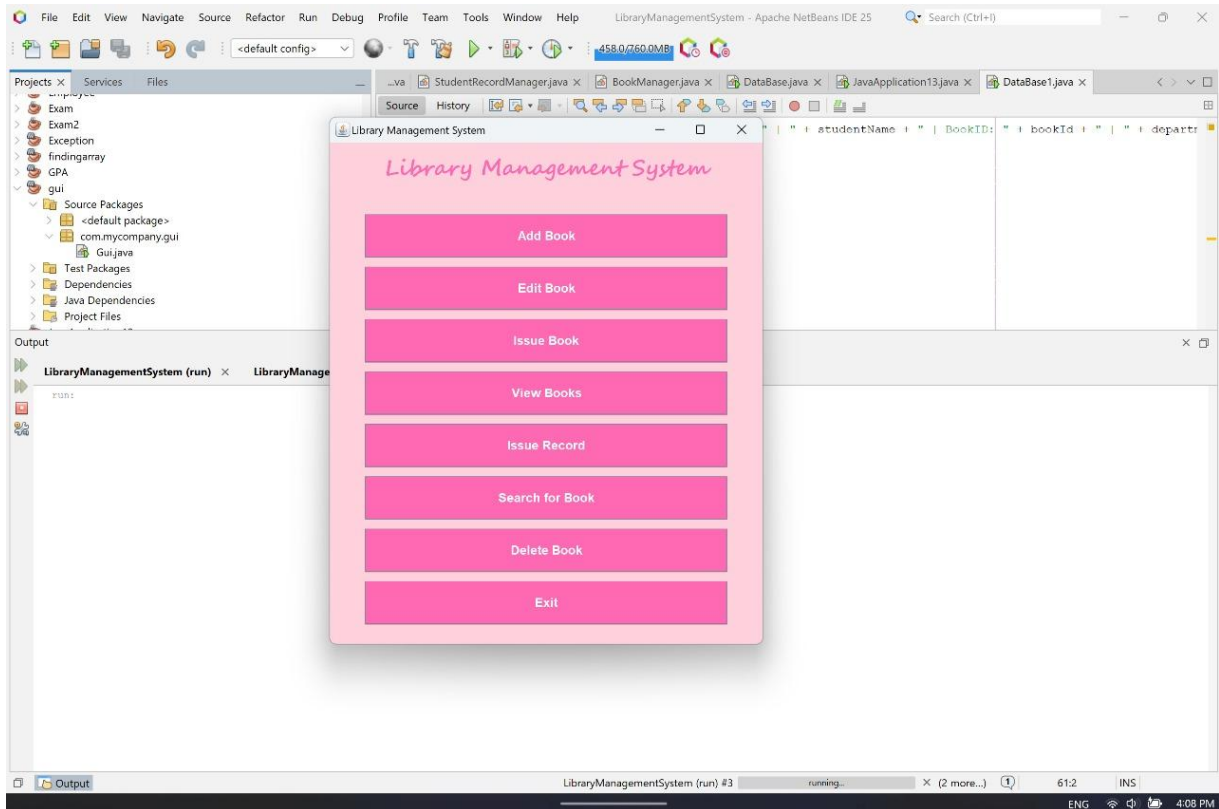
- Receives action events and responds to them (when a certain button is clicked, the program runs the code)



GUI and Database Usage

This section describes the GUI and Database Usage in the project.

1. GUI:



This is a graphical user interface (GUI) for a Library Management System, built using Java Swing. Here's a brief explanation of its layout and functionality:

1. JFrame — The Main Window

- This is the outer window titled "Library Management System".
- It's the container that holds everything else (buttons, labels, panels).

2. JLabel — The Title Text

- The title "Library Management System" at the top is likely created using a JLabel.

3. JButton — The Pink Buttons

- Buttons like "Add Book", "Edit Book", etc., are JButton components.

4. JPanel — Grouping Components

- The buttons are arranged vertically, in a JPanel with a GridLayout.
- This helps in organizing components neatly.

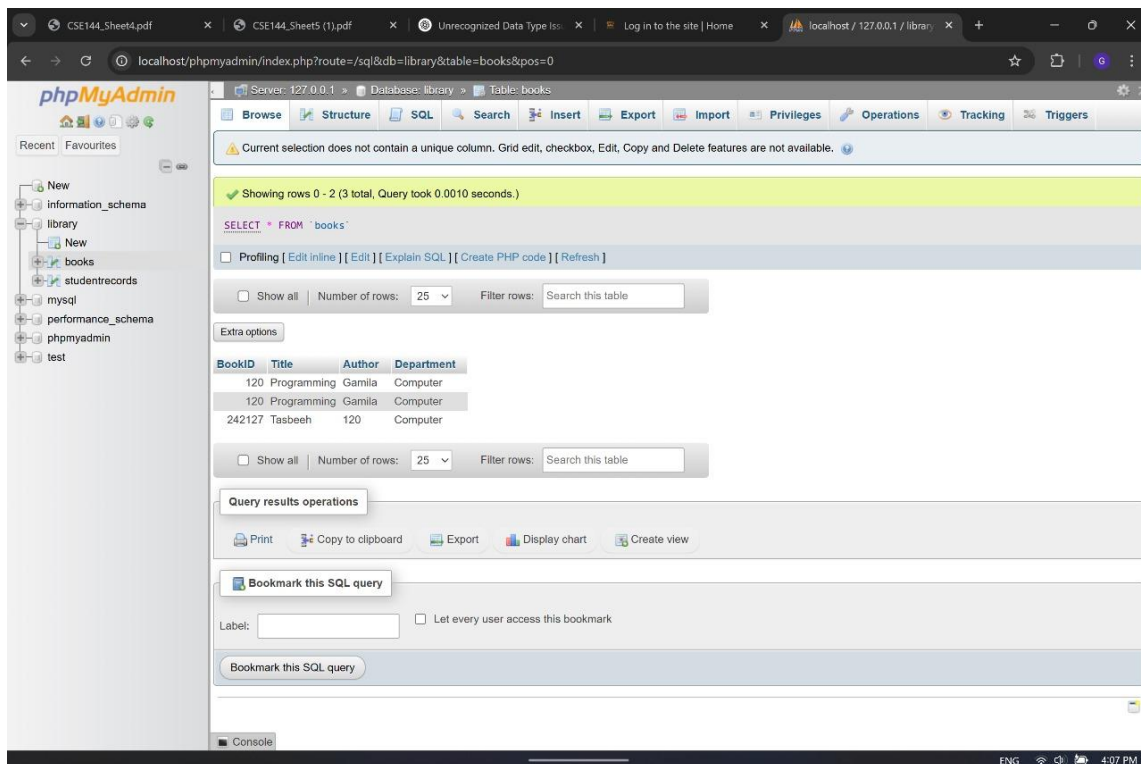


5. Exit Button Functionality

- To make the "Exit" button actually close the application:

2. Database:

a. Books Table:



The screenshot shows the phpMyAdmin interface for a database named 'library'. The 'books' table is selected, and its structure and data are displayed. The table has four columns: BookID, Title, Author, and Department. The data shows three rows of books.

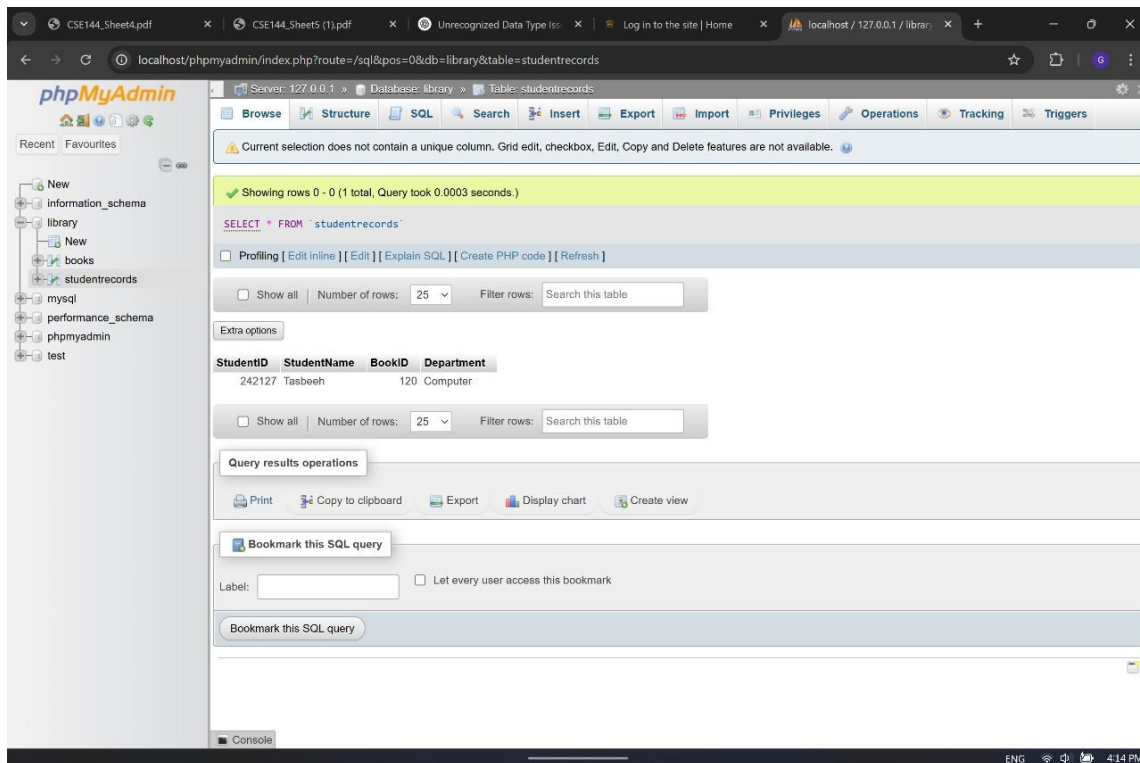
BookID	Title	Author	Department
120	Programming	Gamila	Computer
120	Programming	Gamila	Computer
242127	Tasbeeh	120	Computer

This table stores information about each book in your library. Each row represents a single book, with the following columns:

- BookID:** A unique identifier for each book.
- Title:** The name of the book.
- Author:** The author of the book.
- Department:** The department associated with the book.



b. *Students Table:*



This table records which students have borrowed which books. Each row represents a borrowing record, with the following columns:

- **StudentID:** A unique identifier for each student.
- **StudentName:** The name of the student.
- **BookID:** The identifier of the book borrowed (should match a BookID in the books table).
- **Department:** The department the student belongs to.

Relationship Between Tables

The *BookID* in the *StudentRecords* table serves as a foreign key that references the *BookID* in the books table. This relationship allows you to determine which book a student has borrowed.

This setup exemplifies a one-to-many relationship: one book can be borrowed by many students over time, but each borrowing record refers to a single book.



Code explaining

1. BookManager Class

```
package librarymanagementsystem;
import java.io.File;
import java.util.ArrayList;
import java.io.FileWriter;
import java.io.IOException;

public class BookManager {

    //Attributes
    private String departement, author, title;
    private int bookID;

    static ArrayList<BookManager> bookList = new ArrayList<>();

    public BookManager(){ //Default Constructor
        departement = author = title = "";
        bookID = 0;
    }

    //Parametrized Constructor
    public BookManager(String departement, String author, String title, int bookID){
        this.departement = departement;
        this.author = author;
        this.title = title;
        this.bookID = bookID;
    }

    //Setters and Getters
    public String getDepartement() {
        return departement;
    }

    public String getAuthor() {
        return author;
    }

    public String getTitle() {
        return title;
    }

    public int getBookID() {
        return bookID;
    }
    public void setBookID(int bookID){
        this.bookID=bookID;
    }

    //Method adds book to a departement and save is to a file
    public void addBook(){
```




```
bookList.add(this); //Adds current book object to the ArrayList

//Saves the book details to a file
try{
    // Create department folder on D: drive if it doesn't exist
    new File("C:/" + departement).mkdir();
    FileWriter f = new FileWriter("C:/" +departement + "/" + bookID + ".txt");
    f.write("Departement: "+departement+"\nTitle: "+title+"\nAuthor: "+author+"\nBook
ID: "+bookID); //Writes book data in the file
    f.close(); //Closes the file after writing
    System.out.println("Book Added to: " +departement+" Departement"); //Confirms to
the user
} catch(Exception e){
    System.out.println("Failed to add book..."); //Displaying a message if saving
fails
}
//Method deletes book from the library
public void deleteBook(){
    for(int i=0 ; i<bookList.size() ; i++){
        if(bookList.get(i).bookID == this.bookID){ //Check if the book is found in the
ArrayList
            bookList.remove(i); //Removes from ArrayList if found
            File bookFile = new File("C:/" + departement + "/" + bookID + ".txt");
            if (bookFile.exists()) {
                bookFile.delete(); // Delete the file
            }//Deletes the file of the book
            System.out.println("Book Deleted: "+this.title); //Confirms to the user
            return; //Exit the method after deleting the book
        }
    }
    System.out.println("Book not found in library.."); //If the book doesn't exist in the
ArrayList
}

//Method edits the data of an existing book
public void editBook(String newTitle, String newAuthor) {
    boolean found = false;

    for (BookManager book : bookList) {
        if (book.bookID == this.bookID) { // Check if the book is found in the ArrayList
            found = true;

            // Delete the old file (removes the old book file from the department folder)
            File oldFile = new File("C:/" + book.departement + "/" + book.bookID + ".txt");
            if (oldFile.exists()) {
                oldFile.delete(); // Delete the old file
            }

            // Update book details (only title and author)
            book.title = newTitle;
            book.author = newAuthor;

            // Ensure the department folder exists on C: drive
            File deptFolder = new File("C:/" + book.departement); // Assuming department
folder already exists in C:/
```



```
if (!deptFolder.exists()) {
    System.out.println("Department folder does not exist.");
    return; // Exit if the department folder doesn't exist
}

// Write updated data to a new file in the department folder on C:/
try {
    // Write updated book details to a new file in the department folder
    FileWriter f = new FileWriter("C:/\" + book.departement + "/" + book.bookID +
".txt");
    f.write("Department: " + book.departement + "\nTitle: " + newTitle +
"\nAuthor: " + newAuthor + "\nBook ID: " + book.bookID);
    f.close(); // Close the file after writing
    System.out.println("Book Edited and saved successfully."); // Confirms to the
user
} catch (IOException e) {
    System.out.println("Failed to edit book..."); // Displaying a message if
saving fails
}
break;
}
}

if (!found) {
    System.out.println("Book not found in library.");
}
}

//Method to search for a book
public static String searchBook(String title, String author, int bookID) {
    for (BookManager book : bookList) {
        if (book.getTitle().equalsIgnoreCase(title) ||
            book.getAuthor().equalsIgnoreCase(author) ||
            book.getBookID() == bookID) { //Return book details if found
            return "Book Found:\nTitle: " + book.getTitle() +
"\nAuthor: " + book.getAuthor() +
"\nDepartment: " + book.getDepartement() +
"\nBook ID: " + book.getBookID();
        }
    }
    return "Book not found in library."; //Return a message to the user if not found
}

public boolean issueBook(int bookId, String studentName, int studentID) {
    for (BookManager book : bookList) {
        if (book.bookID == bookId) {
            // Book found - create and issue record
            StudentRecordManager issued = new StudentRecordManager(studentName, studentID,
bookId, book.departement);
            return issued.issueRecord(); // true if successful
        }
    }
    return false; // book not found
}

//Method to view all available books in library
```



```
public static String viewBooks() {
    if (bookList.isEmpty()) { //Checking the ArrayList
        return "No books available."; //Returning a message if not found
    }

    String result = "";
    for (BookManager book : bookList) {
        result += "\nDepartement: " + book.departement + //Returning all books' details
            "\nTitle: " + book.title +
            "\nAuthor: " + book.author +
            "\nBook ID: " + book.bookID + "\n";
    }
    return result;
}

}
```

2. GUI Interface

```
package librarymanagementsystem;

// Importing necessary libraries for GUI and event handling
import javax.swing.*;
import java.awt.*; // Corrected import (was incorrect: java.awt.ntsyste*)
import java.awt.event.ActionEvent;
import java.awt.event.ActionListener;

public class LibraryGui {
    public static void main(String[] args) {
        // Ensures GUI is created on the Event Dispatch Thread
        SwingUtilities.invokeLater(() -> new LibraryGui().displayMainMenu());
    }

    public void displayMainMenu() {
        // Create the main application window
        JFrame f = new JFrame("Library Management System");
        f.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        f.setSize(500, 600); // Set window size

        // Define color scheme for the GUI
        Color pastelPink = Color.decode("#FFD1DC");
        Color darkPink = Color.decode("#FF69B4");
        Color white = Color.WHITE;

        // Create a panel with a grid layout for buttons
        JPanel p = new JPanel(new GridLayout(8, 1, 10, 10));
        p.setBackground(pastelPink);
        p.setBorder(BorderFactory.createEmptyBorder(20, 40, 20, 40));

        // Create all buttons
        JButton addBookButton = createStyledButton("Add Book", darkPink, white);
        JButton editBookButton = createStyledButton("Edit Book", darkPink, white);
        JButton issueBookButton = createStyledButton("Issue Book", darkPink, white);
        JButton viewBooksButton = createStyledButton("View Books", darkPink, white);
        JButton issueRecordButton = createStyledButton("Issue Record", darkPink, white);
        JButton searchButton = createStyledButton("Search for Book", darkPink, white);
        JButton deleteBookButton = createStyledButton("Delete Book", darkPink, white);
    }
}
```



```
 JButton exitButton = createStyledButton("Exit", darkPink, white);

// Add buttons to the panel
p.add(addBookButton);
p.add(editBookButton);
p.add(issueBookButton);
p.add(viewBooksButton);
p.add(issueRecordButton);
p.add(searchButton);
p.add(deleteBookButton);
p.add(exitButton);

// Add panel to frame
f.add(p);

// Create a label as a title
JLabel titleLabel = new JLabel("Library Management System", SwingConstants.CENTER);
titleLabel.setForeground(new Color(255, 105, 180));
titleLabel.setFont(new Font("Segoe Script", Font.BOLD, 24));
titleLabel.setBorder(BorderFactory.createEmptyBorder(10, 10, 10, 10));

// Create wrapper panel with border layout to add title and button panel
JPanel wrapper = new JPanel(new BorderLayout());
wrapper.setBackground(pastelPink);
wrapper.add(titleLabel, BorderLayout.NORTH);
wrapper.add(p, BorderLayout.CENTER);

// Set the content of the frame and make it visible
f.setContentPane(wrapper);
f.setVisible(true);

// Button actions: Add Book
addBookButton.addActionListener(e -> {
    String department = JOptionPane.showInputDialog("Enter Department: ");
    String author = JOptionPane.showInputDialog("Enter Author: ");
    String title = JOptionPane.showInputDialog("Enter title: ");
    String ID = JOptionPane.showInputDialog("Enter Book ID ");
    if (department == null || author == null || title == null || ID == null ||
        department.isEmpty() || title.isEmpty() || author.isEmpty() || ID.isEmpty()) {
        return;
    }
    try {
        int bookID = Integer.parseInt(ID);
        BookManager book = new BookManager(department, author, title, bookID);
        book.addBook();
        JOptionPane.showMessageDialog(null, "Book added successfully");
    } catch (NumberFormatException ex) {
        JOptionPane.showMessageDialog(null, "Invalid. Book ID must be a number");
    }
});
```



```
// Exit button closes the application
exitButton.addActionListener(e -> System.exit(0));
}

// Helper method to create a styled JButton
private JButton createStyledButton(String text, Color bgColor, Color fgColor) {
    JButton button = new JButton(text);
    button.setBackground(bgColor);
    button.setForeground(fgColor);
    button.setFocusPainted(false);
    button.setFont(new Font("Arial", Font.BOLD, 14));
    return button;
}

}
```

3. StudentRecordManager Class

```
package librarymanagementsystem;
import java.io.File;
import java.io.FileWriter;
import java.io.IOException;
import java.util.ArrayList;

public class StudentRecordManager {

    // Attributes
    private String studentName;
    private int studentID;
    private int bookID;
    private String departement;

    // List to store issued records
    static ArrayList<StudentRecordManager> issueList = new ArrayList<>();

    // Default Constructor
    public StudentRecordManager() {
        studentName = departement = "";
        studentID = bookID = 0;
    }

    // Parameterized Constructor
    public StudentRecordManager(String studentName, int studentID, int bookID, String department)
    {
        this.studentName = studentName;
        this.studentID = studentID;
        this.bookID = bookID;
        this.departement = department;
    }

    // Setters and Getters
```



```
public String getStudentName() {
    return studentName;
}

public int getStudentID() {
    return studentID;
}

public int getBookID() {
    return bookID;
}

public String getDepartment() {
    return departement;
}

// Method to issue a book and save record to file
public boolean issueRecord() {
    issueList.add(this); // Add to in-memory list

    try {
        // Create "IssuedBooks" folder on C: drive if it doesn't exist
        File issuedBooksFolder = new File("C:/IssuedBooks");
        if (!issuedBooksFolder.exists()) {
            issuedBooksFolder.mkdirs();
        }

        // Save record to a file
        FileWriter save = new FileWriter("C:/IssuedBooks/" + bookID + ".txt");
        save.write("Student Name: " + studentName + "\n" +
            "Student ID: " + studentID + "\n" +
            "Book ID: " + bookID + "\n" +
            "Department: " + departement + "\n");
        save.close();

        return true; // success
    } catch (IOException e) {
        return false; // failed
    }
}
```

4. Database Handler

```
package librarymanagementsystem;

// Importing necessary Java SQL classes
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.SQLException;
import java.sql.* ;

public class DataBase1 {
    public static void main(String[] args) {
        // Database connection details
    }
}
```



```
String url = "jdbc:mysql://localhost:3306/library"; // Database URL
String user = "root"; // MySQL username
String password = ""; // MySQL password (empty if not set)

try {
    // Load the MySQL JDBC driver
    Class.forName("com.mysql.cj.jdbc.Driver");

    // Establish the database connection
    Connection conn = DriverManager.getConnection(url, user, password);

    // Retrieve and print the contents of the "books" table
    System.out.println("Book Table:");
    String booksQuery = "SELECT * FROM books";
    Statement stmt1 = conn.createStatement();
    ResultSet booksRs = stmt1.executeQuery(booksQuery);

    // Print each row in the "books" table
    while (booksRs.next()) {
        int bookId = booksRs.getInt("BookID");
        String title = booksRs.getString("Title");
        String author = booksRs.getString("Author");
        String department = booksRs.getString("Department");

        // Print the book details in a formatted manner
        System.out.println(bookId + " | " + title + " | " + author + " | " +
department);
    }

    // Retrieve and print the contents of the "studentrecords" table
    System.out.println("\nStudent Records Table:");
    String studentQuery = "SELECT * FROM studentrecords";
    Statement stmt2 = conn.createStatement();
    ResultSet studentRs = stmt2.executeQuery(studentQuery);

    // Print each row in the "studentrecords" table
    while (studentRs.next()) {
        int studentId = studentRs.getInt("StudentID");
        String studentName = studentRs.getString("StudentName");
        int bookId = studentRs.getInt("BookID");
        String department = studentRs.getString("Department");

        // Print the student record details in a formatted manner
        System.out.println(studentId + " | " + studentName + " | BookID: " + bookId +
" | " + department);
    }

    // Close all open resources to prevent memory leaks
    booksRs.close();
    studentRs.close();
    stmt1.close();
    stmt2.close();
    conn.close();

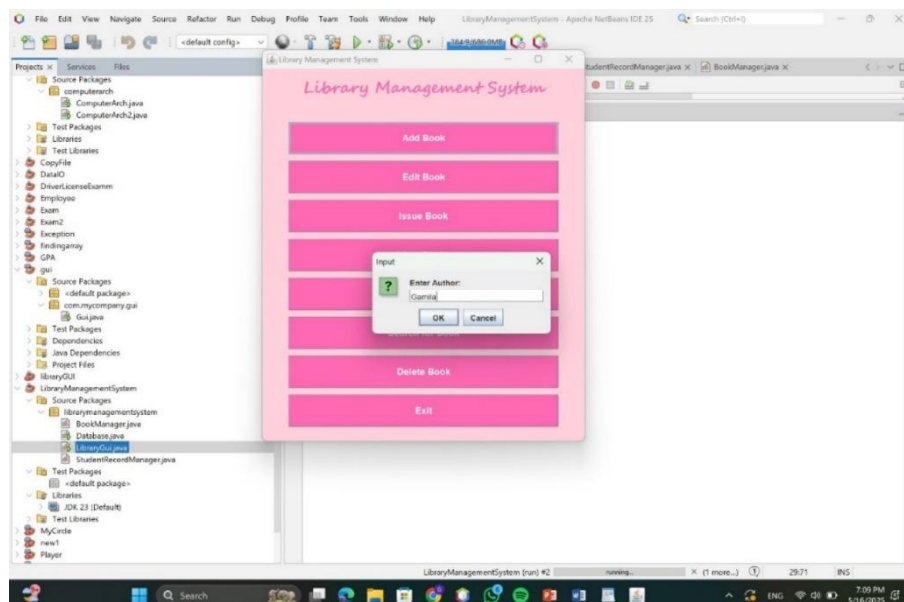
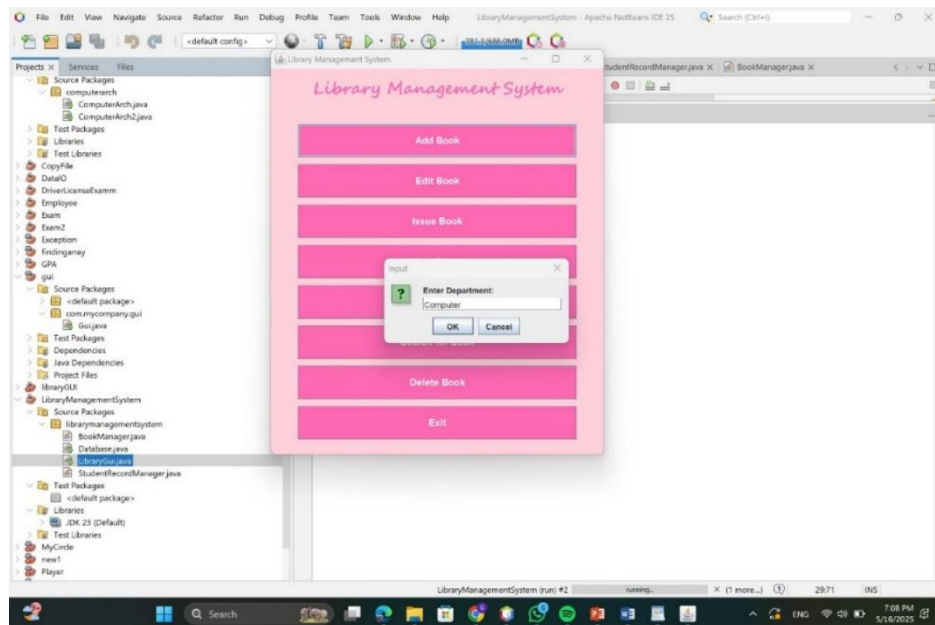
    // Print a completion message
    System.out.println("\nDone.");
} catch (ClassNotFoundException e) {
```

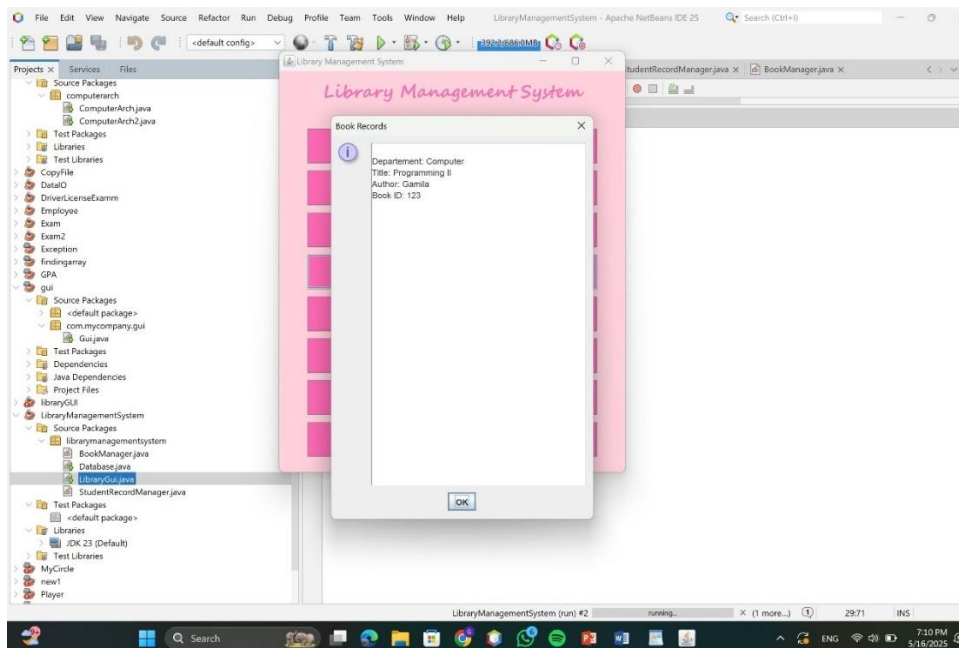
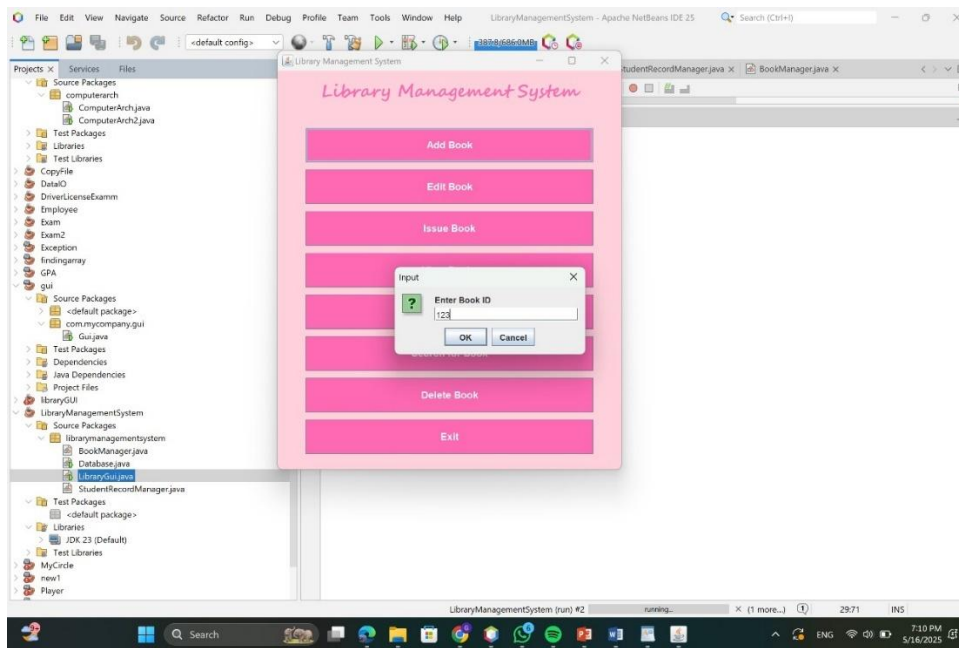



```
// Handle the case where the JDBC driver is not found
System.out.println("JDBC Driver not found.");
e.printStackTrace();
} catch (SQLException e) {
// Handle SQL exceptions
System.out.println("SQL error.");
e.printStackTrace();
}
}
```

Output and results

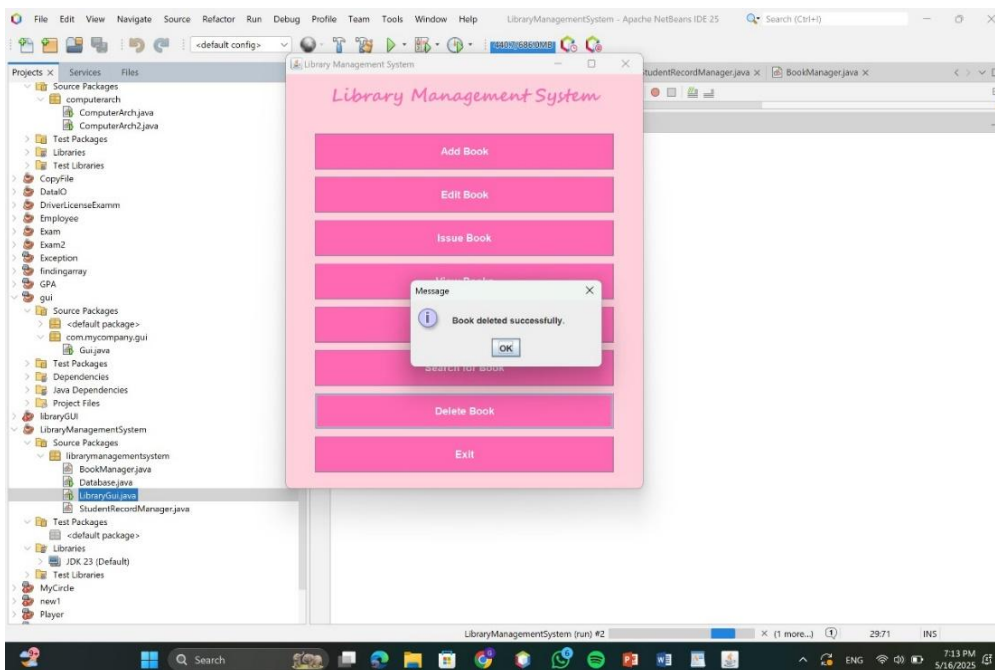
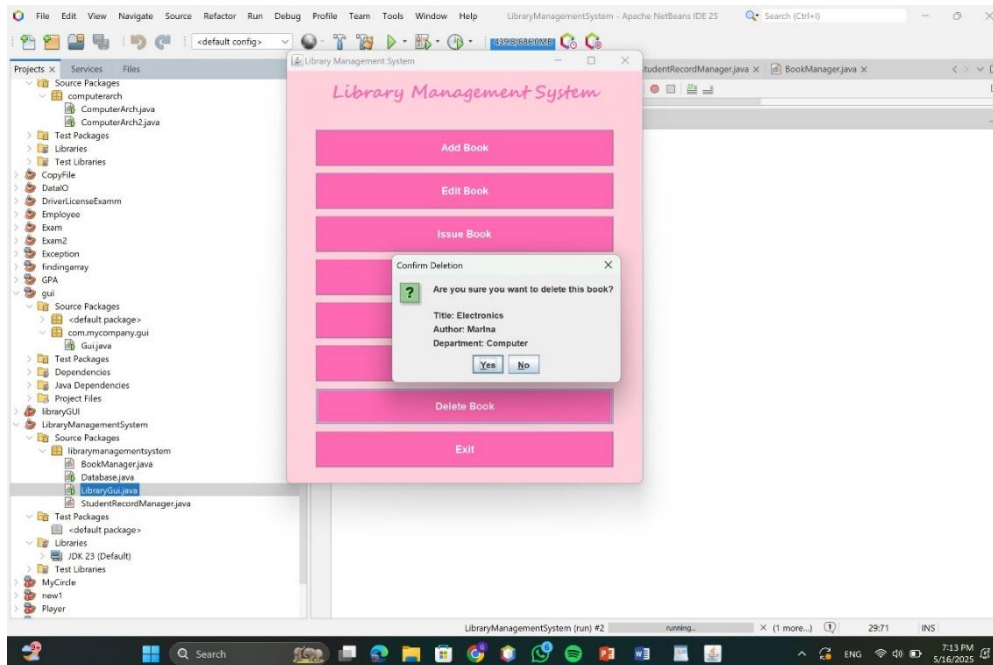
1. Add Book





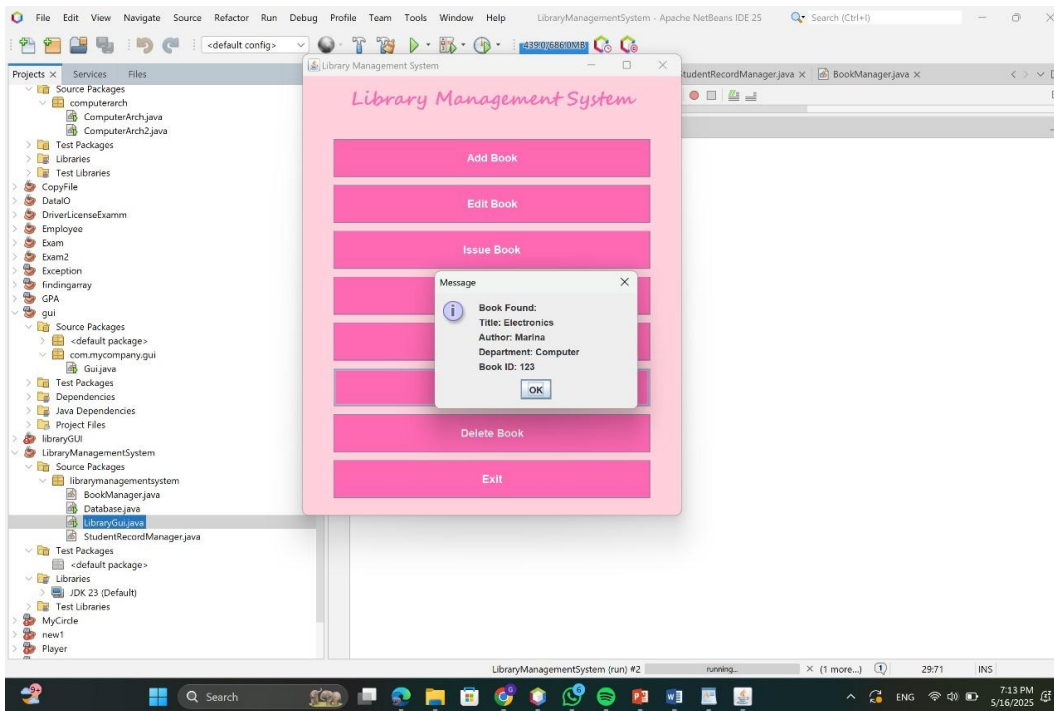
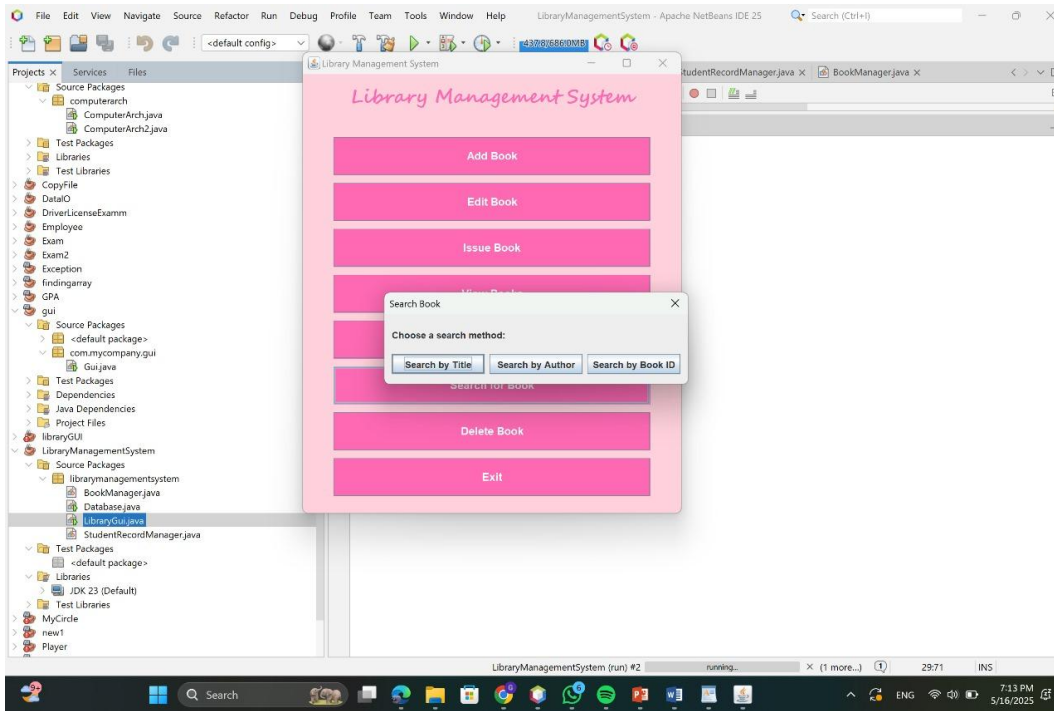
- A new book was added to the book records after clicking on “Add Book”
- The user was asked to enter the book department, book name, author name , and book ID

2. Delete Book



- A book was deleted from the computer department
- A confirmation message appears to the user before any action is taken

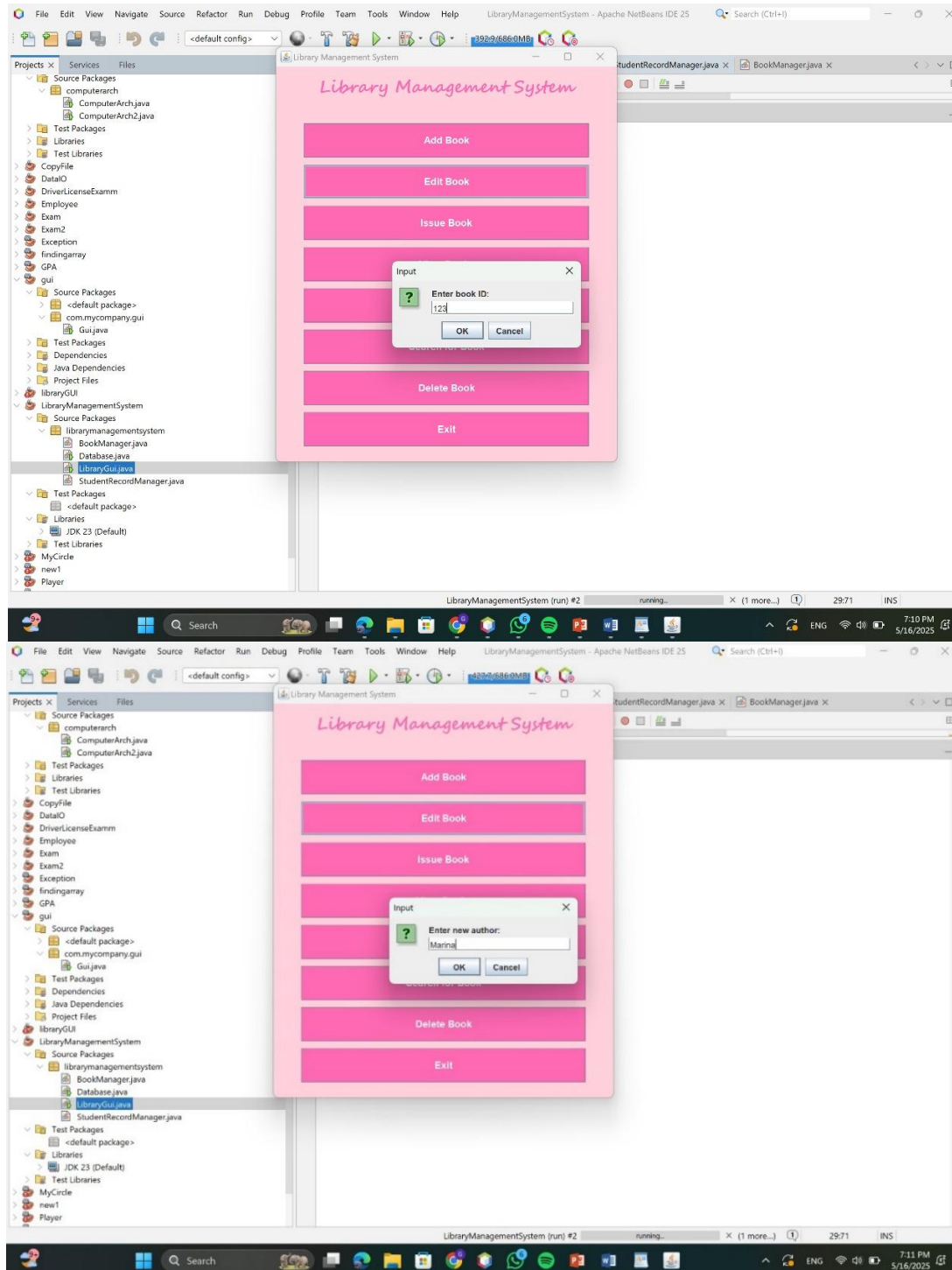
3.Search Book

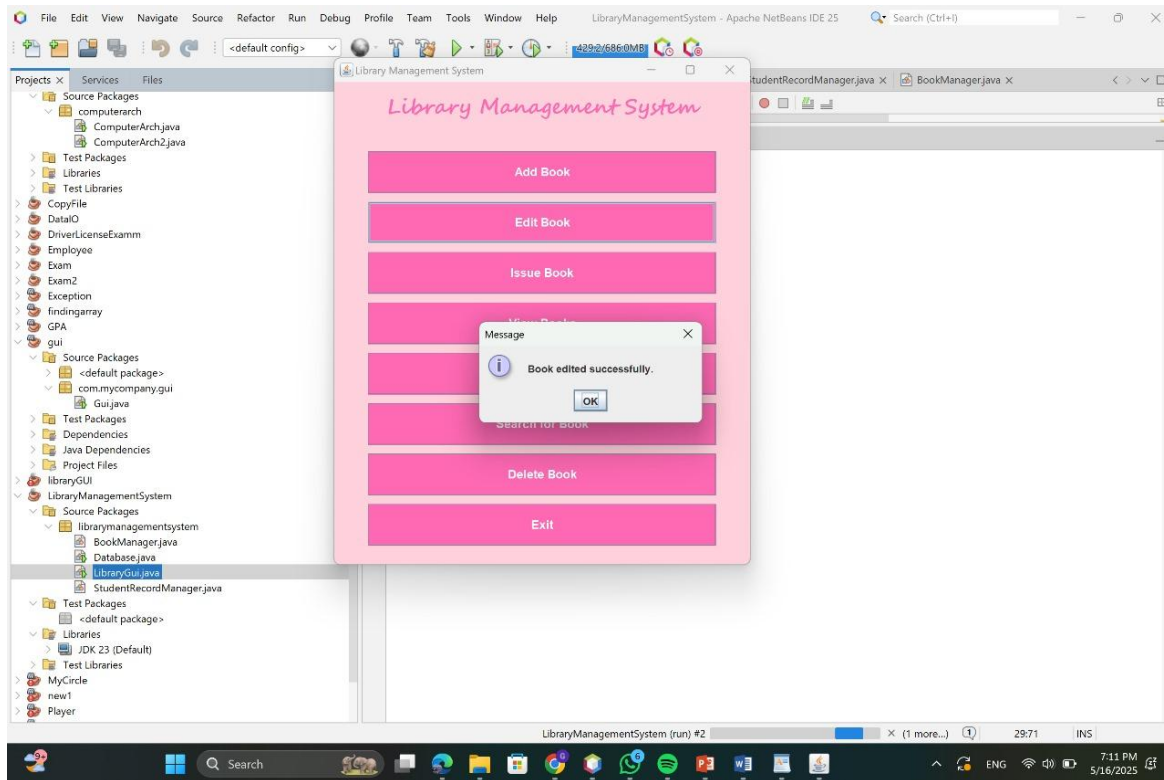


- The user is asked to choose a search method
- After choosing a search method, a message then appears to the user confirming whether or not the book has been found



4.Edit Book

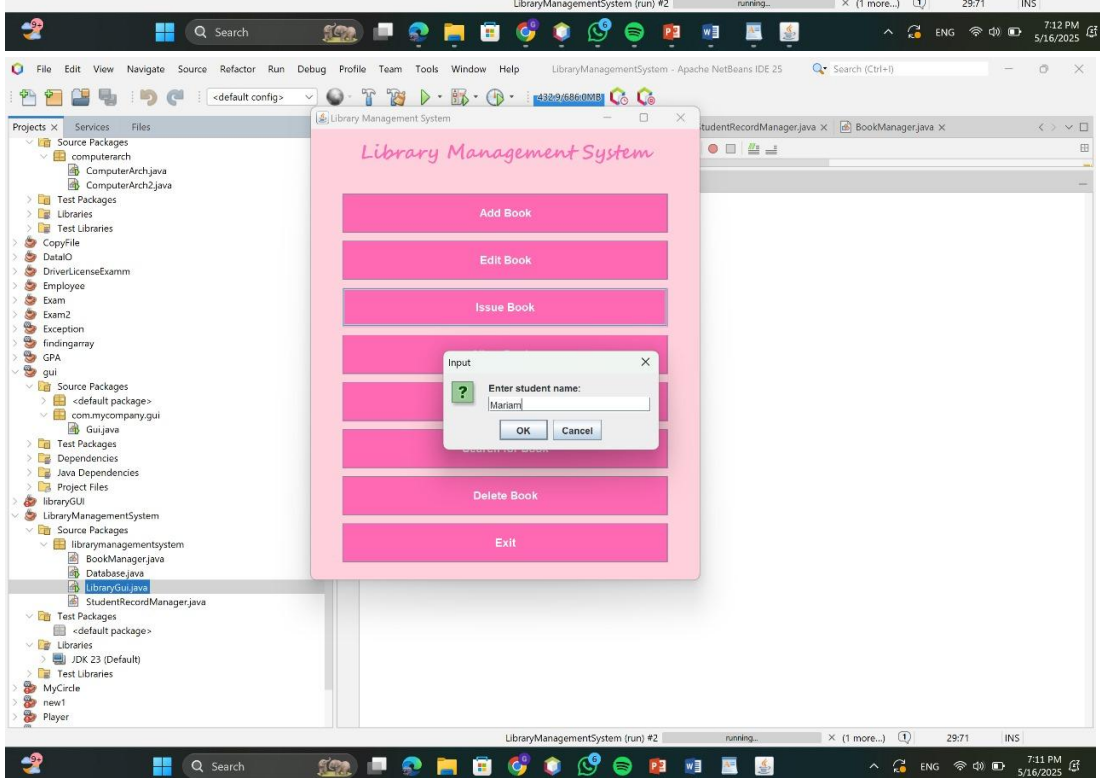
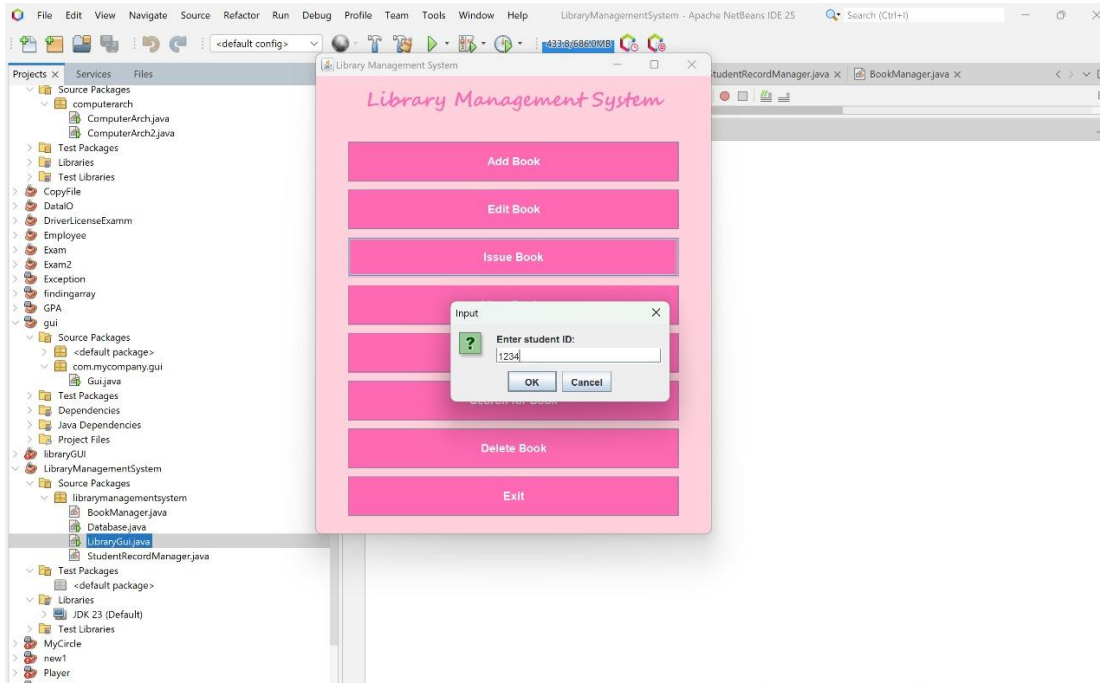


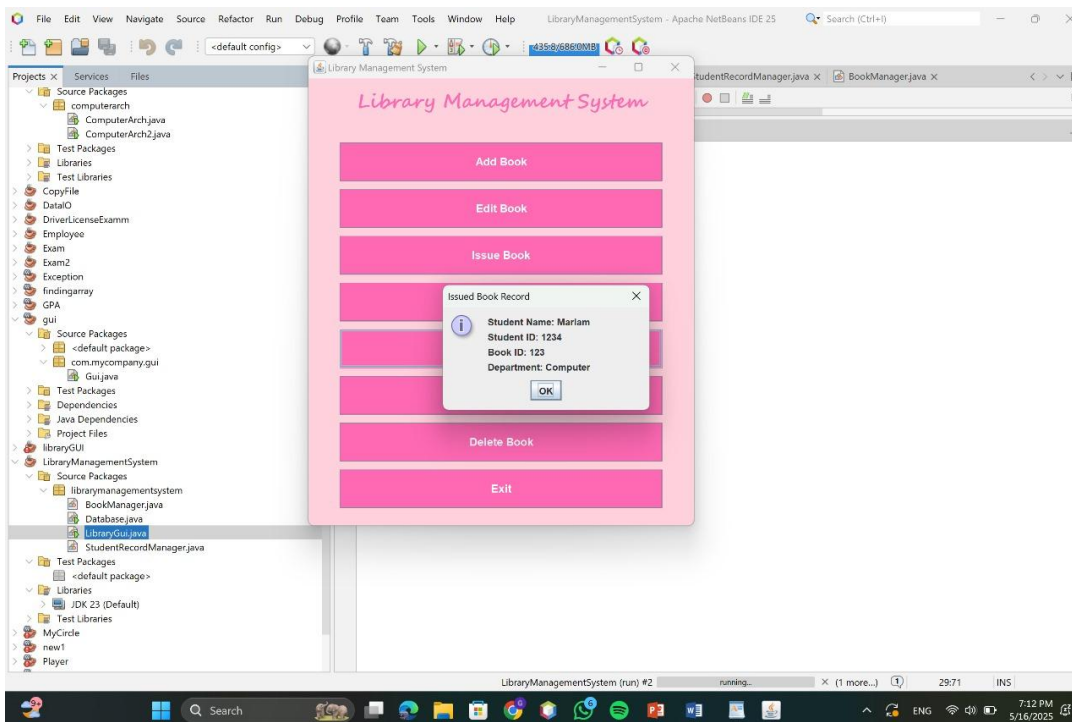
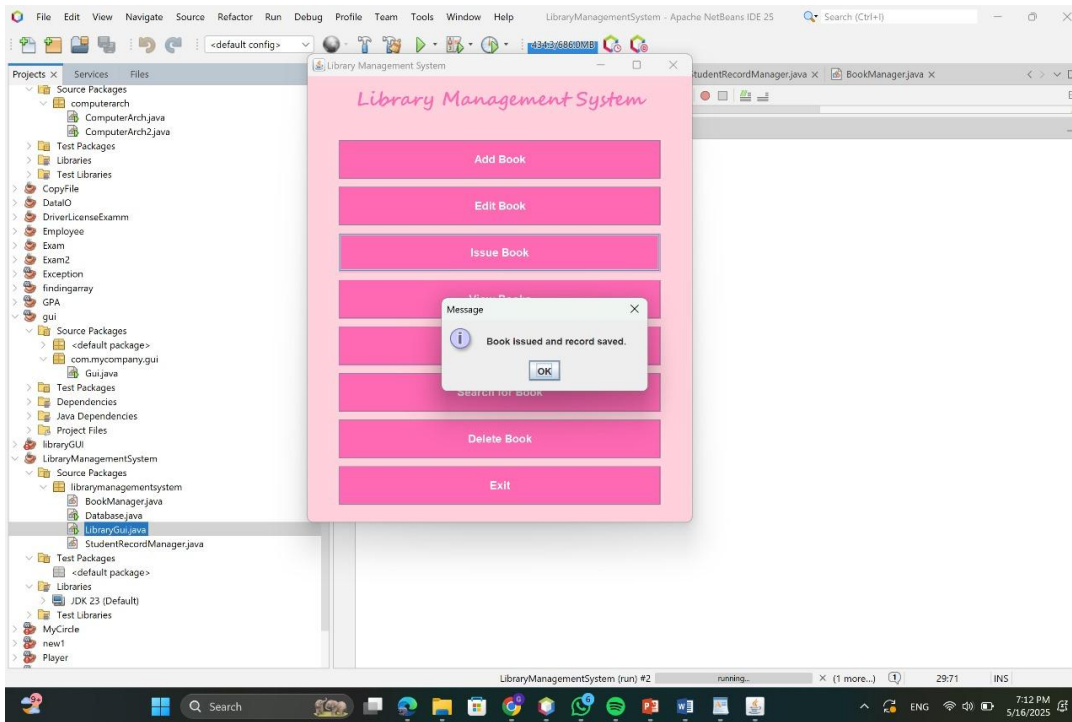


- The user is asked to enter the new author's name and book ID



5.Issue Book





- The user is asked to enter the student ID as well as the student's name
- The issued record is then saved and its information appears to the user on the screen

6. Database

```

> Dependencies
> Java Dependencies
> Project Files

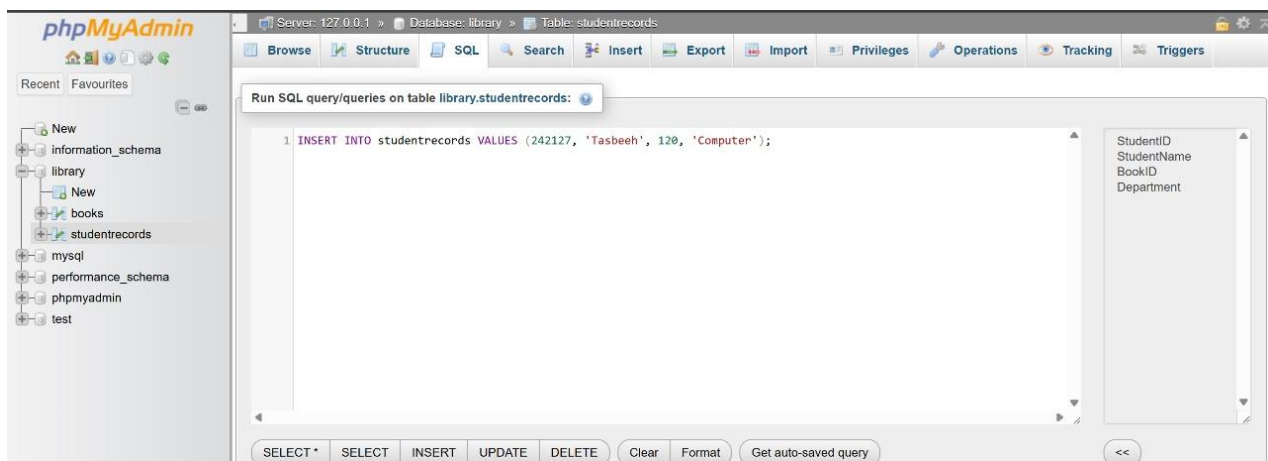
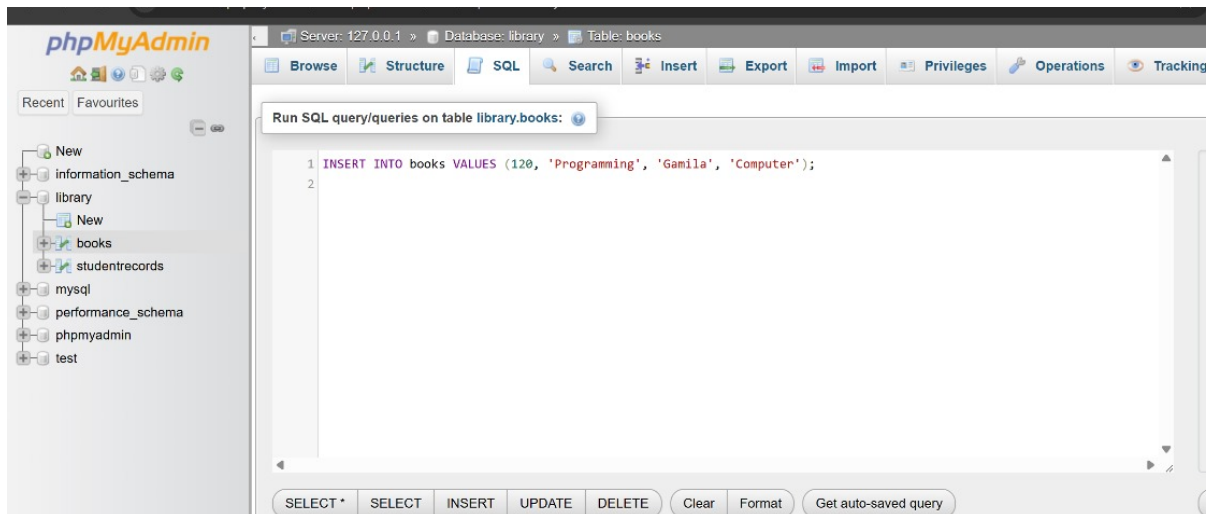
Output
LibraryManagementSystem (run) x LibraryManagementSystem (run) #2 x LibraryManagerer

run:
Book Table:
120 | Programming | Camila | Computer
120 | Programming | Camila | Computer
242127 | Tasbeeh | 120 | Computer

Student Records Table:
242127 | Tasbeeh | BookID: 120 | Computer

Done.
BUILD SUCCESSFUL (total time: 0 seconds)

```





References

1. FileWriter Class in Java
<https://www.geeksforgeeks.org/filewriter-class-in-java/>
2. Java Swing GUI
https://www3.ntu.edu.sg/home/ehchua/programming/java/J4a_GUI.html
3. Database Connectivity with MySQL
<https://www.geeksforgeeks.org/java-database-connectivity-with-mysql/>